

CS 300

Project One

Elijah Bastien

2/14/26

Milestone 1

Data Structure

// Define a structure to hold course data

Structure Course:

String courseNumber

String title

List of Strings prerequisites

Load data into vector

Function loadDataStructure(String csvPath):

Create a Vector of Course objects named 'courseList'

Open file at csvPath

If file fails to open:

Display "File Error: Could not open the data file."

Return empty list

While file has lines to read:

String line = read next line from file

List tokens = Split(line, ",") // Parsing the line by commas

// Criteria: Check for at least 2 parameters (Number and Title)

If tokens.size < 2:

Display "Format Error: Line must have at least a Number and Title."

Continue to next line

// Create course object and assign data

Course currentCourse

currentCourse.courseNumber = tokens[0]

currentCourse.title = tokens[1]

// Add prerequisites if they exist (index 2 and beyond)

For i from 2 to tokens.size - 1:

currentCourse.prerequisites.add(tokens[i])

// Store course in the vector

courseList.push_back(currentCourse)

Close file

// Criteria: Validate that every prerequisite listed exists in the file

For each course in courseList:

For each prereqID in course.prerequisites:

Boolean exists = False

For each matchCourse in courseList:

If prereqID == matchCourse.courseNumber:

exists = True

If exists == False:

Display "Error: Prerequisite " + prereqID + " does not exist in the curriculum."

Return courseList

Search and Print

Function printCourseInformation(Vector courseList, String searchID):

Boolean found = False

// Loop through the vector to find the specific course

For each course in courseList:

If course.courseNumber == searchID:

Display course.courseNumber + ", " + course.title

Display "Prerequisites: "

If course.prerequisites is empty:

Display "None"

Else:

For each p in course.prerequisites:

Display p + " "

found = True

Break

If found == False:

Display "Course not found: " + searchID

Milestone 2

Pseudocode

Design pseudocode to define how the program opens the file, reads the data from the file, parses each line, and checks for file format errors.

```
METHOD LoadCourses(fileName)
```

```
OPEN fileName FOR reading
```

```
IF fileName is not open:
```

```
PRINT "Error: Could not open file"
```

```
RETURN
```

```
// Pass 1: Initial Parsing and Basic Format Check
```

```
WHILE NOT end of file:
```

```
READ line
```

```
SPLIT line by delimiter (",") INTO tokens
```

```
IF tokens.size < 2:
```

```
PRINT "Error: Invalid format on line " + line
```

```
CONTINUE to next line
```

```
// Create Course object (detailed in Section 2)
```

```
// Add to temporary list for Pass 2 validation
```

```
END WHILE
```

```
END METHOD
```

Design pseudocode to show how to create course objects and store them in the appropriate data structure.

```
// Define the data structure
```

```
STRUCT Course:
```

string courseNumber

string courseTitle

list prerequisites

METHOD StoreInDataStructure(allLines):

CREATE HashTable table

FOR EACH line IN allLines:

NEW Course course

course.courseNumber = tokens[0]

course.courseTitle = tokens[1]

// Handle optional prerequisites

FOR i FROM 2 TO tokens.size:

course.prerequisites.ADD(tokens[i])

// Pass 2: Logical Validation

FOR EACH prereq IN course.prerequisites:

IF prereq NOT FOUND in file course list:

PRINT "Error: Prerequisite " + prereq + " does not exist."

// Insert into Hash Table

table.INSERT(course.courseNumber, course)

RETURN table

END METHOD

Design pseudocode that will print out course information and prerequisites.

METHOD PrintCourseInfo(table, searchNumber):

// Retrieve course using the Hash Table key

Course course = table.SEARCH(searchNumber)

IF course is NOT null:

PRINT course.courseNumber + ": " + course.courseTitle

PRINT "Prerequisites: "

IF course.prerequisites is EMPTY:

PRINT "None"

ELSE:

FOR EACH prereq IN course.prerequisites:

PRINT prereq + " "

ELSE:

PRINT "Course number " + searchNumber + " not found."

END METHOD

Milestone 3

Pseudocode

// Struct to hold course information

Struct Course:

```
courseNumber: String
courseTitle: String
prerequisites: List of Strings
```

```
// Node structure for the Binary Search Tree
```

```
Struct Node:
```

```
courseData: Course
left: Pointer to Node
right: Pointer to Node
```

```
// Main function to load and process course data
```

```
Function LoadCourses(fileName):
```

```
    BST = Empty Tree
```

```
    // 1. Open and Validate File
```

```
    File = Open(fileName)
```

```
    If File does not exist:
```

```
        Print "Error: Could not open file"
```

```
        Return NULL
```

```
    // 2. Read and Parse Data
```

```
    While File is not EOF (End of File):
```

```
        Line = ReadLine(File)
```

```
        Tokens = Split(Line, ",") // Assuming comma-separated
```

```
    // Validate format: At least course ID and title
```

If Size(Tokens) < 2:

 Print "Error: Invalid line format - " + Line

 Continue // Skip this line

// Create Course Object

NewCourse = New Course

NewCourse.courseNumber = Tokens[0]

NewCourse.courseTitle = Tokens[1]

// Add prerequisites if they exist

For i from 2 to Size(Tokens) - 1:

 Add Tokens[i] to NewCourse.prerequisites

// Insert Course Object into BST

Insert(BST, NewCourse)

File.Close()

// 3. Post-processing: Validate Prerequisites

ValidatePrerequisites(BST)

Return BST

// Function to insert a course into the BST

Function Insert(Node, NewCourse):

 If Node is NULL:

```
Return New Node(NewCourse)
```

```
If NewCourse.courseNumber < Node.courseData.courseNumber:
```

```
    Node.left = Insert(Node.left, NewCourse)
```

```
Else If NewCourse.courseNumber > Node.courseData.courseNumber:
```

```
    Node.right = Insert(Node.right, NewCourse)
```

```
Return Node
```

```
// Function to validate that all prerequisites exist in the BST
```

```
Function ValidatePrerequisites(Node):
```

```
    If Node is NULL:
```

```
        Return
```

```
    // Check prerequisites for current course
```

```
    For Each PrereqID in Node.courseData.prerequisites:
```

```
        If Search(BST, PrereqID) is NULL:
```

```
            Print "Error: Prerequisite " + PrereqID + " not found for " +  
Node.courseData.courseNumber
```

```
    // Recursively check children
```

```
    ValidatePrerequisites(Node.left)
```

```
    ValidatePrerequisites(Node.right)
```

```
// Function to print course information (In-order traversal)
```

```
Function PrintSortedCourses(Node):
```

If Node is NULL:

Return

PrintSortedCourses(Node.left)

// Print current course info

Print Node.courseData.courseNumber + ": " + Node.courseData.courseTitle

Print "Prerequisites: " + Node.courseData.prerequisites

PrintSortedCourses(Node.right)

Menu Pseudocode

Function MainMenu():

choice = 0

While choice != 9:

Display "1. Load Data Structure"

Display "2. Print Sorted Course List"

Display "3. Print Course Information"

Display "9. Exit"

Input choice

Switch choice:

Case 1:

// Load data based on chosen data structure

dataStructure = LoadCourses("courses.csv")

Case 2:

// For Tree: PrintSortedCourses(root)

```
// For Vector/Hash: Sort then print
```

```
PrintSortedList(dataStructure)
```

Case 3:

```
Input courseID
```

```
PrintCourseInfo(dataStructure, courseID)
```

Case 9:

```
Display "Exiting Program"
```

Worst-Case Running Time Analysis (Loading Data)

Data Structure	Loading Complexity	Rationale
Vector	$O(n)$	Linear time to read n lines and insert into the vector.
Hash Table	$O(n)$	Assuming a good hash function, average insertion is $O(1)$, total $O(n)$. Worst case is $O(n^2)$ if all keys collide.
Tree	$O(n \log n)$	Average insertion is $O(\log n)$. In a highly unbalanced tree, this could become $O(n^2)$.

Analysis of Data Structures

Structure	Advantages	Disadvantages
Vector	Simple to implement; fast insertion ($O(1)$).	Searching is slow ($O(n)$); requires manual sorting before printing alphabetically.

Structure	Advantages	Disadvantages
Hash Table	Extremely fast search ($O(1)$ average).	Does not maintain order; requires extra steps to sort for output.
Tree	Maintains data in sorted order automatically; fast searching ($O(\log n)$ average).	More complex to implement than a vector or hash table.

Recommendation

I recommend using a Binary Search Tree for the ABCU advising program. While a Hash Table offers the fastest search times for individual course lookups ($O(1)$), the requirements specifically ask to print the courses in alphanumeric order (Option 2). A Binary Search Tree inherently stores data in a sorted manner based on the courseNumber (key). This allows us to print the sorted list in $O(n)$ time using an in-order traversal, whereas a Hash Table would require extracting all elements and sorting them ($O(n \log n)$) every time the list is printed. The $O(\log n)$ search time is also highly efficient for looking up individual course information